

Text Processing

Python 文本处理讲义

清华大学统计与数据科学系

俞声 | 2026.4.15

1. 何时需要文本处理

文本处理（Text Processing）是数据科学和自然语言处理中的基础技能，广泛应用于以下场景：

- 数据清洗（Data Cleaning）
- 数据操作（Data Manipulation）
- 信息抽取（Information Extraction）
- 为文本挖掘与 NLP 做数据准备

2. 常见文本处理操作

文本处理涵盖范围广泛，常见操作包括：

- 字符计数（Counting the number of characters）
- 字符串连接（Concatenating）
- 字符串分割（Splitting）
- 提取子串（Extracting substrings）
- 替换子串（Substituting substrings）
- 去除空白（Trimming）
- 填充对齐（Padding）

- 模式匹配（Pattern matching）——这是一个大类别
- 词干提取/词形还原（Stemming / Lemmatizing）

3. Python 文本处理的三大支柱

str	re	pandas.str
内置字符串方法 适用于简单需求 不支持模式匹配	正则表达式模块 基于正则表达式的 复杂模式匹配	向量化文本处理 适用于 Series 和 DataFrame 支持正则表达式

4. Python 内置字符串方法（str）

Python 的 str 类型提供了丰富的内置方法，适用于无需模式匹配的简单文本处理任务。

方法	说明
count	返回子字符串在字符串中非重叠出现的次数
endswith	若字符串以指定后缀结尾，返回 True
startswith	若字符串以指定前缀开头，返回 True
join	用字符串作为分隔符，将序列中的字符串连接
index	返回子字符串第一次出现的起始索引；未找到则抛出 ValueError
find	返回子字符串第一次出现的位置；未找到则返回 -1
rfind	返回子字符串最后一次出现的位置；未找到则返回 -1
replace	将字符串中所有匹配的子字符串替换为另一个字符串
strip / rstrip / lstrip	去除两侧/右侧/左侧的空白字符（包含换行符）
split	使用指定分隔符将字符串分割为子字符串列表
lower / upper	将字母字符转换为小写/大写
casefold	转为小写，并将特定区域字符组合转换为统一比较形式
ljust / rjust	左对齐或右对齐，并用空格（或指定字符）填充至

5. 正则表达式 (Regular Expressions)

正则表达式（简称 regex 或 regexp）是一种文本模式，由**正则表达式引擎（regex engine）**处理，用于对给定字符串进行模式匹配。正则表达式被大多数编程语言支持，语法基本一致。

示例：邮箱匹配正则表达式

```
\b[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}\b

import re
import sys

def main():
    while True:
        try:
            regex = input("\nEnter your regex: ") # 输入正则表达式
            if regex == "":
                continue
            input_string = input("Enter input string to search: ") # 输入要搜索的字符串
            try:
                pattern = re.compile(regex) # 编译正则表达式
                matcher = pattern.finditer(input_string) # 查找所有匹配项
                found = False
                for match in matcher:
                    print(f'I found the text "{match.group()}" starting \
                        at index {match.start()} and ending at index {match.end()}.')
                    found = True
                if not found:
                    print("No match found.")
            except re.error as e:
                print(f'Regex error: {e}')
        except (KeyboardInterrupt, EOFError):
            # 当用户按 Ctrl+C 或 Ctrl+Z (Ctrl+D) 时退出程序
            print("\nExiting...")
            sys.exit(0)
```

这段只有直接运行时执行，作为模块import时是不执行的。

```
if __name__ == "__main__":
    main()
```

5.1 字面量 (Literals)

最基础的正则表达式由单个字面字符组成（如 a），它匹配字符串中所有该字符的出现位置。

正则表达式默认区分大小写，可以通过 re.IGNORECASE 标志忽略大小写。

```
re.split('a', 'Star Wars') # 区分大小写
re.split('A', 'Star Wars', flags=re.IGNORECASE) # 忽略大小写
```

5.2 元字符 (Metacharacters)

以下字符在正则表达式中具有特殊含义，如需匹配其字面字符，需用 \ 进行转义：

- 反斜杠 \、脱字符 ^、美元符号 \$
- 点号 .、竖线 |、问号 ?
- 星号 *、加号 +
- 圆括号 ()、方括号 []、花括号 {}

在 Python 中，推荐使用原始字符串 (raw string) 避免双重转义：

```
re.split('\\+', '1+1')    # 双重转义
re.split(r'\+', '1+1')    # 推荐：使用原始字符串
```

5.3 字符类 (Character Classes)

字符类 匹配多个字符中的一个，用方括号 [] 表示。

- gr[ae]y 匹配 "gray" 或 "grey"，但不匹配 "graay"
- 用连字符表示范围：[0-9]、[a-zA-Z]
- 在 [后面加 ^ 表示取反类：q[^x] 匹配字母 q 后面跟着一个不是 x 的字符，可匹配 "qu"（出现在 "question" 中），但不匹配 "Iraq"

5.4 简写字符类 (Shorthand Character Classes)

简写	含义	等价 (re.ASCII)
\d	数字字符	[0-9]
\w	单词字符	[A-Za-z0-9_]
\s	空白字符	[\t\r\n\f\v] #空格、制表符、换行、换页、垂直制表符
\D	非数字字符	[^d]
\W	非单词字符	[^w]
\S	非空白字符	[^s]

注意：不同正则引擎的行为可能存在差异！

5.5 通配符 (Wildcard)

点号 `.` 匹配除换行符 (`\r` 和 `\n`) 以外的任意单个字符。

正则 `a.b` 能匹配 `"axb"`、`"a@b"`、`"a b"` (中间是任意一个字符)。
但无法匹配跨行的情况, 比如 `"a\nb"`, 因为 `.` 不匹配 `\n`。

5.6 锚点 (Anchors)

锚点匹配的是位置, 而非字符本身:

锚点	含义
<code>^</code>	字符串/行的开始位置
<code>\$</code>	字符串/行的结束位置
<code>\b</code>	单词边界
<code>\B</code>	非单词边界的位置

5.7 量词 (Quantifiers)

量词指定前面元素的重复次数, 分为

贪婪 (Greedy): 吃剩下的所有字符, 如果匹配上了就直接结束, 失败时回吐字符重试

懒惰 (Reluctant): 吃尽量少的字符, 每次只匹配最少次数, 如果不满足则再多吃一个, 逐步扩展

独占 (Possessive): 吃剩下的所有字符, 一旦匹配就绝不回吐, 不进行回溯 (Backtracking)

贪婪	懒惰	独占	含义
<code>X?</code>	<code>X??</code>	<code>X?+</code>	X 出现一次或零次
<code>X+</code>	<code>X+?</code>	<code>X++</code>	X 出现一次或多次
<code>X*</code>	<code>X*?</code>	<code>X*+</code>	X 出现零次或多次
<code>X{n}</code>	<code>X{n}?</code>	<code>X{n}+</code>	X 恰好出现 n 次
<code>X{n,}</code>	<code>X{n,}?</code>	<code>X{n,}+</code>	X 至少出现 n 次
<code>X{n,m}</code>	<code>X{n,m}?</code>	<code>X{n,m}+</code>	X 出现 n 到 m 次

贪婪 Greedy

懒惰 Reluctant

独占 Possessive

...

已吞入 当前尝试 回吐（回溯） 最终匹配 未参与

贪婪 (Greedy)：先一口气吃掉所有能吃的字符，然后看整体模式是否匹配。失败了就从末尾**逐个吐出一个字符**再重试，直到匹配成功或全部吐光。

方向：向右尽量多吃 → 向左逐步回吐

正则：a.*b 输入：aXbYb

匹配过程

③ 需要 b 但已到末尾，失败

a X b Y b

末尾之后找不到 b，匹配失败
→ 触发回溯：吐出最后一个字符 "b"

← 上一步

下一步 →

步骤 3/5

贪婪 Greedy

懒惰 Reluctant

独占 Possessive

...

已吞入 当前尝试 回吐（回溯） 最终匹配 未参与

懒惰 (Reluctant)：先只吃最少字符（往往0个或1个），然后看整体模式是否匹配。失败了就再**多吃一个**，逐步扩展，直到匹配成功或超出范围。

方向：向右最少开始 → 逐步向右扩展

正则：a.*?b 输入：aXbYb

匹配过程

④ 索引2 = "b"，匹配成功！ 匹配成功

a X b Y b

"b" 匹配成功！
最终匹配：a X b (索引 0-2)
停止扩展，尽量短

← 上一步

下一步 →

步骤 4/4

贪婪 Greedy

懒惰 Reluctant

独占 Possessive

...

已吞入

当前尝试

回吐 (回溯)

最终匹配

未参与

独占 (Possessive)：和贪婪一样先全部吃掉，但**绝不回吐**——吃进去的字符永远不释放，不做任何回溯。所以一旦贪婪地匹配导致整体失败，它就直接报失败，不再尝试其他可能。

方向：向右全部吃 → 不回头，直接成功或失败

正则：a.*+b 输入：aXbYb

匹配过程

③ 需要 b 但末尾无字符

a

X

b

Y

b

末尾之后找不到 b，匹配失败

但独占模式：绝不回吐，不做回溯

← 上一步

下一步 →

步骤 3/4

5.8 交替与分组 (Alternation and Grouping)

- cat|dog food 匹配 "cat" 或 "dog food"
- (cat|dog) food 匹配 "cat food" 或 "dog food"
- 圆括号 () 创建捕获组 (capturing group)
- 可用反向引用 \1、\2 等引用已匹配的捕获组内容

示例：

```
# Set(Value)? 匹配 "Set" 或 "SetValue"
# ([abc])=\1 匹配 "a=a"、"b=b"、"c=c"
# (dog){3}    匹配 "dogdogdog"
# dog{3}     匹配 "doggg" (仅 g 重复 3 次)
```

5.9 环视 (Lookaround)

类型	语法	示例说明	解释
正向前瞻	q(?=u)	匹配 "question" 中的 q，不匹配 "Iraq" 中的 q	q 后面跟着 u 才匹配
负向前瞻	q(?!u)	匹配 "Iraq" 中的 q，不匹配 "question" 中的 q	q 后面不跟 u 才匹配

正向后顾	(?<=a)b	匹配 "abc" 中的 b	b 前面是 a 才匹配
负向后顾	(?<!a)b	不匹配 "abc" 中的 b	b 前面不是 a 才匹配

注意：前瞻支持完整正则表达式，大多数应用的后顾只允许固定长度的表达式。

?= 是正则表达式中的前瞻断言（Lookahead）。

语法：(?=...)

它的作用是：向前查看，确认某个内容存在，但不消耗字符。

(?=.*[A-Z]) 意思是：从当前位置往后看，必须存在至少一个大写字母，但匹配完之后"指针"不移动，继续检查后面的条件。

(?=...) 适合"必须包含某东西"

直接写字符类适合"必须是什么样的字符"

例如测试密码是否符合规范 `r^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#%&*])[^\s]{8,}$'`

5.10 re 模块常用方法

方法	说明
findall	以列表形式返回字符串中所有非重叠匹配项
finditer	与 findall 类似，但返回迭代器
match	从字符串开头匹配模式，成功返回 Match 对象，否则返回 None
search	扫描整个字符串，返回第一个匹配的 Match 对象
split	在每个匹配位置将字符串分割为片段
sub / subn	替换所有（sub）或前 n 个（subn）匹配项；可用 \1、\2 等引用捕获组 以 <code>re.sub(r'([a-z0-9])([A-Z])', r'\1_\2', s1)</code> 为例： • \1 代表第一个捕获组 <code>([a-z0-9])</code> 匹配到的内容 • \2 代表第二个捕获组 <code>([A-Z])</code> 匹配到的内容 • 中间插入 <code>_</code> 这个例子的“替换”指的是：把匹配到的内容，替换成"原内容+下划线"的新内容

5.11 Match 对象

search、match、fullmatch 返回 Match 对象，finditer 的每个迭代项也是 Match 对象。

若无匹配，Match 对象为 None，使用前务必先判断！


```

match = re.search(r'(\w+) (\w+)', 'hello world')
if match:
    print(match.group())      # 'hello world' (整体匹配)
    print(match.group(1))    # 'hello' (第1组)
    print(match.group(2))    # 'world' (第2组)
    print(match.start(1))    # 0 (第1组起始索引)
    print(match.end(2))      # 11 (第2组结束索引)
    print(match.span(1))     # (0, 5) (第1组范围)

```

5.12 正则表达式速度注意事项

正则表达式对于简单任务速度很快，但以下情况可能导致性能下降：

- 在大输入上使用复杂模式（涉及大量回溯）
- 使用大型词典进行匹配

6. pandas 字符串方法 (pandas.str)

pandas.str 提供了对 Series 和 DataFrame 中字符串数据的向量化文本处理，内置正则表达式支持。

6.1 pandas.str 常用方法

方法	说明
cat	可选分隔符下按元素连接字符串
contains	返回布尔数组，表示各元素是否包含指定模式/正则
count	统计指定模式的出现次数
extract	使用带分组的正则表达式提取字符串，结果为 DataFrame（每组一列）
endswith	等价于对每个元素执行 <code>x.endswith(pattern)</code>
startswith	等价于对每个元素执行 <code>x.startswith(pattern)</code>
findall	对每个字符串计算所有匹配项列表
get	索引每个元素（取第 i 个元素）
len	计算每个字符串的长度
lower / upper	大小写转换，等价于对每个元素执行 <code>x.lower()</code> / <code>x.upper()</code>

match	对每个元素执行 re.match，返回 True 或 False
pad	在字符串左侧、右侧或两侧添加空白
replace	替换指定模式/正则的出现
slice	对 Series 中每个字符串进行切片
split	按分隔符或正则表达式分割字符串
strip / rstrip / lstrip	去除两侧/右侧/左侧空白（含换行符）
repeat	复制值（如 s.str.repeat(3) 等价于 x * 3）

6.2 优先使用 pandas.str 而非 re

在 DataFrame 中处理文本时，应优先使用 pandas.str 方法而非手动调用 re 方法。

推荐：向量化，速度快

```
df['col'].str.replace(r'\d+', 'NUM',
                    regex=True)
```

不推荐：逐一处理，速度慢

```
df['col'].apply(lambda x:
                re.sub(r'\d+', 'NUM', x))
```

7. 综合代码示例

7.1 RegexpTestHarness — 正则表达式测试工具

以下代码演示了如何用 Python 的 re 模块构建一个交互式正则表达式测试工具：

```
import re
import sys

def main():
    while True:
        try:
            regex = input('\nEnter your regex: ')
            if regex == '':
                continue
            input_string = input('Enter input string to search: ')
            try:
                pattern = re.compile(regex)          # 编译正则表达式
                matcher = pattern.finditer(input_string) # 查找所有匹配项
                found = False
                for match in matcher:
                    print(f'Found "{match.group()}" at [{match.start()},
{match.end()})')
                    found = True
            except re.error:
                print('Invalid regex')
        except KeyboardInterrupt:
            break
```

```
        if not found:
            print('No match found.')
        except re.error as e:
            print(f'Regex error: {e}')
    except (KeyboardInterrupt, EOFError):
        print('\nExiting...')
        sys.exit(0)

if __name__ == '__main__':
    main()
```

7.2 pandas.str 实践示例

```
import pandas as pd

df = pd.DataFrame({'text': ['Hello World', 'foo123bar', 'test@example.com']})

# 检测是否包含数字
df['has_digit'] = df['text'].str.contains(r'\d')

# 提取所有数字
df['digits'] = df['text'].str.findall(r'\d+')

# 替换所有数字为 NUM
df['masked'] = df['text'].str.replace(r'\d+', 'NUM', regex=True)

# 转为小写
df['lower'] = df['text'].str.lower()

# 按 @ 分割
df['parts'] = df['text'].str.split('@')
```

8. 参考资料

- Python 内置字符串方法: https://wesmckinney.com/book/data-cleaning#tbl-table_string_methods
- Python re 模块方法: https://wesmckinney.com/book/data-cleaning#tbl-table_regex_method
- pandas Series 字符串方法: https://wesmckinney.com/book/data-cleaning#tbl-table_vec_string
- 正则表达式教程 (Oracle): <https://docs.oracle.com/javase/tutorial/essential/regex/index.html>
- Python re 模块官方文档: <https://docs.python.org/3/library/re.html>